Truth — AI Workflow Rules

Document: `ai-workflow-rules.md`
Status: Operational enforcement guidelines
Purpose: Define mandatory AI coding agent rules, development approach, scope boundaries, implementation rules, escalation procedures, protected file policies, documentation synchronization, testing requirements, change safety, and verification checklists for the Truth project.

---

1. Purpose and Authority

You are an AI coding agent working on the Truth software project. Follow these rules for every task. Treat them as mandatory engineering requirements, not optional recommendations. Build Truth incrementally from its approved specifications. Preserve existing work, respect architectural boundaries, implement only the authorised scope, and verify every change.

Order of Authority

When determining what to implement, follow this strict hierarchy:

1. Explicit instructions for the current task.
2. Approved architecture decisions and documented project constraints.
3. `project-overview.md` [cite: 1].
4. `architecture.md` [cite: 2].
5. The relevant detailed Product Requirements Document (PRD).
6. Tests and existing implementation, provided they do not contradict an approved requirement.
7. Existing comments and informal assumptions.

- If two authoritative instructions conflict, stop the affected work, identify the conflict, and request a decision. Do not silently choose whichever instruction is easiest to implement.
- Do not interpret a future capability described in the project documents as permission to implement it in the current task.

---

2. Overall Development Approach

Rule 2.1: Work from specifications

- Read the relevant project documents before modifying code.
- Identify the requirement, expected behaviour, architectural owner, dependencies, acceptance criteria, and required tests before implementation.
- Do not start coding from a vague feature description when the necessary behaviour and boundaries remain undefined.

Rule 2.2: Implement incrementally

- Complete one bounded implementation unit at a time.
- Keep each unit small enough to understand, review, test, and revert independently where practical.
- Prefer a small working vertical slice over a large collection of disconnected modules.
- Do not implement multiple future phases simply because their requirements appear in the same document.

Rule 2.3: Preserve the architecture

- Follow the responsibilities defined in `architecture.md` [cite: 2].
- Place new code in the module that owns the responsibility. Use established interfaces rather than bypassing module boundaries.
- Do not introduce circular dependencies, duplicate sources of truth, hidden global state, or direct database mutations that bypass required application controls.
- Do not replace an architectural component with a simpler shortcut without explicit approval.

Rule 2.4: Inspect before changing

- Inspect the actual repository before assuming that a file, package, service, component, or test exists.
- Read the relevant source files, configuration, tests, and callers before changing a public interface.
- Do not assume that the repository matches a proposed folder structure or that earlier documentation describes the current implementation accurately.

Rule 2.5: Prefer the smallest correct change

- Implement the minimum change that fully satisfies the approved acceptance criteria.
- Reuse established utilities and patterns when they are appropriate.

- Do not add abstractions, dependencies, configuration systems, background workers, services, or frameworks without a concrete requirement.
- Do not perform unrelated cleanup or refactoring while implementing a feature.

---

3. Task Scoping Rules

Rule 3.1: Define one implementation unit

Before coding, record:

- Unit ID or task name.
- Requirement being implemented.
- Expected behaviour.
- Files expected to change.
- Dependencies and prerequisites.
- Acceptance criteria.
- Tests required.
- Explicit exclusions.
  (If the project already has a task or backlog system, use its identifiers and conventions.)

Rule 3.2: Keep changes inside the approved scope

- Modify only files necessary to complete the current unit.
- Do not change neighbouring features, public interfaces, dependency versions, build systems, data models, authentication mechanisms, or architectural contracts unless the task requires those changes and they are authorised.
- If an additional change becomes necessary, explain why it is required and identify its impact before proceeding.

Rule 3.3: Do not speculate

Do not invent:

- Product behaviour that is not specified.
- API endpoints or response fields.
- Database fields or relationships without a requirement.
- Permissions or user roles.
- External service capabilities.
- Configuration values or secrets.
- Test results.

- Existing functionality that has not been verified.
  (When a requirement is missing, use the ambiguity procedure in Section 6.)

Rule 3.4: Do not silently expand scope

- Do not turn a single task into a larger redesign.
- Do not add features because they may be useful later.
- Do not implement future Truth capabilities merely because the current architecture anticipates them.
- Record future improvements separately from the current implementation.

Rule 3.5: Preserve existing changes

- Check the working tree and relevant branch state before editing.
- Do not overwrite, revert, delete, or reformat unrelated user changes.
- Do not use destructive Git commands or reset operations to make the repository easier to work with.
- Do not create commits, push changes, merge branches, or publish releases unless the current task explicitly authorises that action.

---

4. When to Split Work into Smaller Steps

Split a task into smaller units before implementation when any of the following conditions apply:

- The task changes more than one independently testable architectural responsibility.
- The task requires coordinated changes across multiple layers, such as the frontend, API, Kernel, and persistence.
- The task introduces a new shared contract or changes an existing public contract.
- The task combines a database migration with application behaviour that can be implemented and verified separately.
- The task includes security-sensitive work such as authentication, authorisation, permissions, credentials, or recovery.
- The task depends on an external integration whose availability or behaviour is uncertain.
- The task has unclear acceptance criteria or unresolved dependencies.
- The task is too large to review its changed files and reason about its failure modes.
- The task requires multiple independent test strategies.
- A failure in one part could leave another part in an inconsistent or unsafe state.

Recommended Sequence When Splitting Work:

1. Resolve blocking requirements and architectural decisions.
2. Define or update the shared contract.

3. Implement the smallest required domain or backend behaviour.
4. Implement persistence and error handling where required.
5. Add or update the user interface or integration.
6. Add integration and end-to-end tests.
7. Update documentation and verify the complete unit.

Adjust the order to respect actual dependencies. Do not create artificial tasks that add overhead without improving isolation, clarity, or verification. Do not leave required interfaces as undocumented placeholders merely to claim that a unit is complete.

---

5. Implementation Rules

Rule 5.1: Follow established contracts

- Validate inputs at system boundaries.
- Return structured outputs and errors that match the approved contracts.
- Keep internal implementation details behind their designated interfaces.
- When changing a contract, identify all affected callers, adapters, persisted records, and tests before making the change.

Rule 5.2: Protect the Truth Kernel

- Route accepted state transitions through the designated Kernel boundary.
- Do not let the Oracle, frontend, external adapter, or background task directly mutate authoritative state in a way that bypasses the Kernel's rules.
- Do not treat a proposed change as committed until the required validation and persistence steps have succeeded.

Rule 5.3: Protect Runtime execution

- Execute only registered components and handlers through approved execution interfaces.
- Enforce task lifecycle rules, timeouts, cancellation, resource limits, and permission checks where applicable.
- Do not report a task as successful merely because it started or returned an unverified response.

Rule 5.4: Protect TIR contracts

- Use the approved, versioned TIR schemas for shared objects.
- Validate data before passing it to another component or executing an operation.

- Do not create a competing representation of an existing canonical object without approval.
- Do not silently discard provenance, identity, schema version, sensitivity, or correlation information required by the contract.

Rule 5.5: Separate reasoning from deterministic computation

- Use explicit, tested procedures for calculations and rule-based operations.
- Do not use generated AI prose as the authoritative result of a deterministic calculation.
- Use the Oracle to interpret, plan, and propose. Use the Compute Engine for supported deterministic computation. Use Runtime and the policy boundary for authorised execution.

Rule 5.6: Protect persistence

- Use the approved persistence interface.
- Use transactions for related changes that must succeed or fail together.
- Handle duplicates, retries, corrections, and interrupted operations explicitly.
- Do not overwrite historical records in a way that conceals prior outcomes.
- Do not delete or alter existing data structures without considering migration, compatibility, recovery, and tests.

Rule 5.7: Protect security boundaries

- Enforce authentication and authorisation on the server.
- Do not treat a hidden frontend control, a supplied user ID, or a model-generated request as proof of permission.
- Do not bypass the policy engine for convenience.
- Do not expose credentials, private records, or sensitive internal details in logs, errors, fixtures, or source code.
- Treat external content and model output as untrusted input.

Rule 5.8: Handle errors explicitly

- Define expected errors at module boundaries.
- Do not swallow exceptions silently.
- Do not replace failures with fake success responses.
- Do not retry operations with external side effects unless the retry policy and duplicate-protection strategy are defined.
- If an operation's outcome is uncertain, report that uncertainty and avoid claiming it succeeded or failed without evidence.

---

6. Missing or Ambiguous Requirements

Rule 6.1: Identify ambiguity explicitly

- When requirements are incomplete, list the exact unknowns.
- Explain which implementation decision depends on each unknown and what could go wrong if the agent guesses.
- Distinguish a blocking ambiguity from a minor implementation detail.

Rule 6.2: Ask before making consequential assumptions

Stop and request clarification when an ambiguity affects:

- Security or permissions.
- Data ownership, privacy, or retention.
- Public API contracts.
- Database schema or migration strategy.
- Core architecture or dependency direction.
- Destructive operations or irreversible changes.
- External side effects.
- Product behaviour that could materially differ from the user's intent.

Do not continue into the affected implementation while the decision remains unresolved. Continue unrelated, independently safe work only when it does not depend on the unresolved decision.

Rule 6.3: Use established decisions for minor gaps

- If an unspecified detail is low-risk, reversible, and covered by existing project conventions, use the established convention.
- Document the assumption in the task report.
- Do not turn a minor assumption into a permanent architectural decision.

Rule 6.4: Do not invent external facts

- If an API, package, service, repository file, or framework behaviour must be verified, inspect the relevant documentation, implementation, or test output.
- If verification is unavailable, state the limitation.
- Do not claim compatibility or functionality based on a name, guess, or generated example.

Rule 6.5: Resolve conflicts before implementation

When project files disagree, report:

- The conflicting statements.
- The files and requirements involved.
- The implementation consequences of each option.
- The recommended resolution.
- The decision required to continue safely.
  (Do not silently edit one specification to make the conflict disappear.)

---

7. Protected Files and Restricted Changes

Rule 7.1: Never modify protected files without explicit authorisation

Do not modify, regenerate, delete, or replace the following without explicit instruction for the current task:

- Generated UI library components.
- Generated design-system files.
- Files generated from Figma, design tools, or other code-generation systems.
- Automatically generated API clients, schemas, or type definitions.
- Lockfiles and dependency manifests when a dependency change is not part of the task.
- Database migration history that has already been applied to a shared or persistent environment.
- Authentication, authorisation, cryptographic, or credential-handling modules outside the authorised scope.
- Deployment workflows, release configuration, or production environment settings.
- Shared configuration files that affect unrelated features.
- Existing user-authored files unrelated to the task.
  (If the repository contains an explicit generated-file header or a protected-file manifest, obey it.)

Rule 7.2: Distinguish generated files from their sources

- When a generated file needs to change, identify its authoritative source and the documented generation procedure.
- Modify the source and regenerate the output only when authorised and when the generation process is known.
- Do not hand-edit generated output as a substitute for changing its source.

- If the source or generation procedure cannot be identified, stop and report the problem.

Rule 7.3: Protect project instructions

- Do not rewrite `project-overview.md` [cite: 1], `architecture.md` [cite: 2], `ai-workflow-rules.md`, detailed PRDs, or architecture decision records as a side effect of implementation.
- When implementation reveals that a specification is wrong or incomplete, report the discrepancy and propose a targeted update.
- Make the update only when authorised by the task or the project's established approval process.

Rule 7.4: Inspect the change set

- Before completing a unit, inspect the full diff.
- Remove accidental formatting churn, unrelated edits, generated noise, temporary debugging code, and unrelated file changes.
- Never delete a file merely because it appears unused without checking its references and obtaining approval when the deletion could affect existing behaviour.

---

8. Documentation Synchronisation

Rule 8.1: Keep implementation and documentation consistent

Update the relevant documentation whenever an approved change affects:

- Public interfaces or API schemas.
- Module responsibilities or dependency direction.
- Database schema or persistence behaviour.
- Configuration or setup instructions.
- Authentication or permission behaviour.
- Task lifecycle or execution guarantees.
- Error handling or recovery behaviour.
- Supported features or known limitations.
  (Do not update unrelated documents simply to make a task appear more comprehensive.)

Rule 8.2: Distinguish proposed from implemented behaviour

Use accurate status language:

- **Planned:** specified but not implemented.

- **Implemented:** code exists but has not necessarily passed verification.
- **Tested:** relevant tests have been executed and their results recorded.
- **Verified:** the defined acceptance criteria have been checked against the implementation.
- **Blocked:** work cannot proceed because a dependency, decision, or required capability is unavailable.
  (Do not label a feature production-ready merely because it passes a unit test.)

Rule 8.3: Keep setup instructions executable

- When changing dependencies, commands, environment variables, migrations, or build steps, update the applicable setup documentation.
- Do not document commands that have not been verified or clearly label them as unverified.
- Never claim that a file, endpoint, module, or command exists without inspecting the repository or verifying it through execution.

Rule 8.4: Record architectural decisions

- When an approved change affects a durable architecture decision, create or update the relevant Architecture Decision Record.
- Include the context, decision, alternatives considered, consequences, and status.
- Do not silently replace a previously accepted decision with a different approach.

---

9. Testing and Verification Rules

Rule 9.1: Test the changed behaviour

- Add or update tests for the behaviour introduced or changed by the task.
- Cover normal behaviour, invalid inputs, important boundary conditions, and expected failure modes.
- Do not rely exclusively on tests that exercise implementation details without verifying the required behaviour.

Rule 9.2: Use the narrowest useful test first

Run tests in this order when applicable:

1. Focused unit tests for the changed module.
2. Relevant contract tests.
3. Integration tests for affected boundaries.
4. Security tests for affected trust boundaries.

5. Frontend or end-to-end tests for affected user journeys.
6. The broader regression suite.
   (Expand the test scope when changes affect shared contracts, core state, security, or multiple modules.)

Rule 9.3: Do not hide failures

- Do not skip failing tests, weaken assertions, or remove coverage merely to obtain a passing result.
- Do not change a test's expected outcome unless the requirement has legitimately changed and the change is authorised.
- If a test fails because of a pre-existing issue, identify it separately and provide the evidence.

Rule 9.4: Do not claim unexecuted verification

- Report the actual test command, whether it ran, and the result.
- If the environment cannot execute a test, report it as not run.
- If a test fails, report the failure and its known cause without claiming the unit is fully verified.

Rule 9.5: Verify migrations and recovery

- For persistence changes, test the relevant schema setup or migration path.
- Where applicable, test restart recovery, transaction failure, duplicate handling, and preservation of existing records.
- Do not test destructive migrations against valuable user data.

---

10. Version Control and Change Safety

- Inspect the current branch and working-tree status before making changes.
- Preserve user modifications and unrelated work.
- Keep the diff focused on the current unit.
- Do not force-push, rewrite shared history, or delete branches without explicit authorisation.
- Do not commit secrets, local databases containing private data, temporary exports, or generated artefacts that are not intended for version control.
- Do not commit or publish code unless the task explicitly authorises it.
- Report when repository access or permissions prevent the requested action.
- Never claim that a change has been pushed, merged, or deployed without confirming the operation's actual result.

---

11. Required Workflow for Every Unit

- **Step 1: Inspect** — Read the relevant specifications, inspect the repository, identify the affected modules, and check existing work.
- **Step 2: Scope** — Write the unit's objective, expected files, dependencies, acceptance criteria, and exclusions.
- **Step 3: Resolve** — Identify missing requirements, conflicts, and architectural risks. Obtain decisions for blocking ambiguities.
- **Step 4: Plan** — Describe the smallest safe implementation sequence and identify the tests required.
- **Step 5: Implement** — Make only the approved changes. Follow existing conventions and the architectural invariants.
- **Step 6: Test** — Run the relevant tests, inspect failures, and fix issues caused by the change.
- **Step 7: Review** — Inspect the full diff, check protected files, verify dependency boundaries, and look for security or data-integrity regressions.
- **Step 8: Synchronise** — Update relevant documentation, schemas, examples, and setup instructions.
- **Step 9: Report** — Provide a concise implementation report containing unit ID/objective, files changed, behaviour implemented, tests executed/results, acceptance criteria satisfied, known limitations, architectural/documentation decisions, and outstanding blockers.
- **Step 10: Stop at the boundary** — Do not begin the next independent unit until the current unit is verified or its remaining blockers are explicitly documented and the project owner authorises the next step.

---

12. Verification Checklist Before Moving to the Next Unit

Confirm every applicable item before declaring a unit complete:

- ☐ The implementation matches the approved requirements.
- ☐ The task remained within its authorised scope.
- ☐ No unresolved blocking ambiguity was guessed away.
- ☐ The correct architectural module owns the new behaviour.
- ☐ Public contracts and schemas are consistent.
- ☐ Input validation and error handling are implemented.
- ☐ Authentication, ownership, and permissions are preserved where relevant.
- ☐ Persistence and state transitions follow the approved boundaries.

- [ ] External operations and background tasks report their actual outcomes.
- [ ] Relevant tests were run and results recorded.
- [ ] Security and failure cases were tested where applicable.
- [ ] Documentation reflects the implementation.
- [ ] Protected or generated files were not changed without authorisation.
- [ ] The complete diff contains no unrelated or accidental changes.
- [ ] No secrets, private data, or unintended generated artefacts were introduced.
- [ ] Known limitations and unverified behaviour are documented.
- [ ] The implementation status is reported honestly.

---

13. Completion Report Format

Use this format at the end of each unit:

- **Unit:** State the unit ID and objective.
- **Changes:** List the files changed and describe the behaviour implemented.
- **Verification:** List the commands executed and their actual results. Distinguish passing, failing, and unrun tests.
- **Acceptance Criteria:** List which criteria passed, failed, or remain unverified.
- **Risks and Limitations:** Report unresolved defects, assumptions, security concerns, and incomplete behaviour.
- **Documentation:** List the documentation updated or explain why no update was required.
- **Next Unit:** Identify the next recommended unit and its prerequisites. Do not begin it automatically if the current unit has unresolved blocking issues.

---

14. Final Rule

Build Truth through controlled, evidence-based engineering. Do not substitute speed for correctness, code volume for progress, generated output for verified implementation, or assumptions for approved requirements. Complete one bounded unit at a time. Preserve the architecture, enforce the security boundaries, keep documentation aligned with code, and report only what has actually been implemented and verified.